2 COMPUTATIONAL COMPLEXITY AND QUANTUM COMPUTING

In this chapter we introduce the basic concepts of quantum computing. We examine the reasons that push us to search for new computational paradigms and why quantum computing appears to be promising.

In order to fully understand the expectations placed on quantum computing, it is first necessary to introduce some elements of computability and complexity. We will therefore examine the foundations of computer science starting from computational models. We will give a definition of a computational problem and see which problems we can solve with a computer. Finally, we will study the computational complexity of problems by classifying them according to the resources required to find a solution.

Having completed this introduction to computer science, we will see how quantum computers fit into this analysis. We will begin by presenting a computational model that allows us to define which problems are solvable on a quantum computer and we will note how this naturally leads us toward quantum-based quantum computers. We will see which complexity classes can be defined considering this new paradigm and we will try to understand what relationships exist between the classes of problems identified with the classical model of computation and those derived from the quantum model.

In the second section of this chapter we will present in a more rigorous way quantum gate-based computers. We will see what is meant by a quantum gate, present the most common gates, and construct a universal set of gates. We will then see what it means to program a quantum gate-based computer by presenting some algorithms of particular interest that seem to suggest that this new computational paradigm is indeed capable of efficiently solving problems that we consider difficult on classical architectures.

The chapter concludes by presenting *Quantum Adiabatic Computing* (QAC), an alternative paradigm to computation based on quantum gates. We will describe the theoretical foundations of this paradigm and show that QAC and quantum gate-based computing are equivalent. We will see that programming a machine that exploits the QAC paradigm means translating the problem we want to solve into a minimization problem solvable through *annealing*. We will describe this technique by drawing a parallel between classical simulated annealing and quantum annealing. Finally, we will focus on the formulation of minimization problems naturally solvable with quantum annealing.

2.1 INTRODUCTION TO COMPUTER SCIENCE

Computer science deals with studying which problems can be solved through a computer and which resources are necessary to find the solution. We can divide these two objectives into two parallel and complementary lines of re-

search. On the one hand, to prove that a problem is solvable it is sufficient to show an algorithm capable of solving that problem, without worrying too much about its complexity. On the other hand, to understand which resources are necessary, we would like to find a lower bound on the operations that must be performed to obtain the solution.

These lines of research are complementary in the sense that ideally, once a lower bound has been identified, we would like to be able to construct an algorithm that matches it. In practice, this is not always possible, both because constructing algorithms that behave well on all instances of a problem is difficult, and because the lower bound of the required operations is not always known, so one must settle for the best algorithm developed so far.

We have referred to an algorithm, intuitively knowing that it is a sequence of operations that allows us to find the answer to a problem. To formally study the characteristics of an algorithm, this intuition is not sufficient; we need something that we can characterize more precisely. We therefore introduce computational models.

2.1.1 Computational models

A computational model allows us to mathematically define the concept of an algorithm. These abstractions were first introduced by Turing and Church to find a solution to Hilbert's *entscheidungsproblem*[1]. We now consider two equivalent computational models: the *Turing machine* and the *circuit model*.

Turing machine

This is the most well-known computational model. Introduced by Turing to formally describe the notion of an algorithm that performs a computational task.

A Turing machine can be seen as the conceptualization of a computer. It is composed of four main elements:

PROGRAM: which allows the encoding of an algorithm;

FINITE STATE CONTROL: unit that acts as the processor;

TAPE: memory of the computer;

READ-WRITE TAPE-HEAD: to interact with the memory.

It is possible to define various versions of the Turing machine. For our discussion we can consider the simplest version, in which the memory tape is unbounded to the right and bounded to the left. The symbols that can be written on the tape are 0, 1, b (the blank symbol) and the special symbol $\triangleright$ that marks the left end of the tape. These symbols form our alphabet Γ; everything that can be represented on the tape consists of finite-length strings Γ^* made up of symbols from the alphabet (except $\triangleright$, which appears only at the beginning of the tape). The finite state control consists of a finite set of states $q1, \ldots, q_m$ plus two special states q_s and q_h that represent,

1 For every mathematical problem, does there exist an algorithm capable of solving it?

respectively, the initial state of the machine and the final (halt) state; the latter is reached when the machine has finished the computation.

A program for the Turing machine consists of program lines; each line is a quintuple $\langle q, x, q', x', s\rangle$ which is interpreted as follows: if the machine is in state q and reads the symbol $x \in \Gamma$ on the tape, then it transitions to state q', writes the symbol $x' \in \Gamma$ on the tape, and moves as indicated by s: to the right $+1$, to the left (-1), or remains stationary (0).

Although the Turing machine model may appear simple, it is capable of computing a very large number of functions; in particular, this model completely captures the notion of computing a function using an algorithm. This thesis was proposed independently by Church and Turing and is known as the *Church–Turing thesis*:

Thesis 1. *The class of functions computable by a Turing machine corresponds exactly to the class of functions which we would naturally regard as being computable by an algorithm.*

This thesis states that the Turing machine is a computational model capable of providing a rigorous definition of the intuitive concept of an algorithm, and is therefore suitable as a foundation for computer science.

The Turing machine we have described can be modified, for example, by imagining having multiple memory tapes; this new machine could be able to solve problems more quickly than the original, but we will always be able to simulate the multi-tape machine with a single-tape machine. This means that even considering multiple tapes does not increase the power of the Turing machine.

Another modification that we can make to this computational model, which will be useful later, is to consider a probabilistic version of the Turing machine: if the machine is in state q and the symbol x is read, a coin is flipped; if the result is heads, the machine transitions to state q_H, otherwise to state q_T. This modification can also be simulated with a Turing machine that explores all possible computation paths one at a time (using exponentially more steps), so once again the set of computable functions is not extended.

UNDECIDABILITY: Computational models, and with them computer science, were developed to answer Hilbert's entscheidungsproblem. Church with the λ-calculus[2] and Turing with the Turing machine showed that the answer is no: there cannot exist an algorithm to decide all the problems of mathematics.

This result clearly divides mathematical problems into two classes: those that can be solved with an algorithm and those so difficult as to be computationally unsolvable. We call the problems belonging to this class *undecidable*. Turing shows that the *halting problem* (the problem of determining whether a Turing machine halts given an input) is, in general, undecidable[3]. In addition to the problems used in the proofs of Church and Turing, there exist many other undecidable problems, for example finding a homomorphism

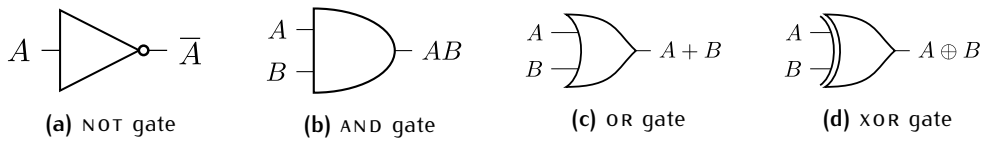
2 Another computational model equivalent to the Turing machine.

3 Church shows that there is no computable function that allows one to determine whether two λ-calculus expressions are equivalent.

between two topological spaces or some problems related to the behavior of dynamical systems.

Circuit model

The circuit model is an alternative model of computation in which we describe an algorithm by implementing it through a circuit made of logic gates. This approach is interesting because it brings us closer to the first quantum gate-based model of computation and gives us the possibility (in Section 2.5) to discuss *reversible* and *irreversible* logic gates.



(a) NOT gate **(b)** AND gate **(c)** OR gate **(d)** XOR gate

Figure 2.1: Logic gate

In Figure 2.1 the most well-known logic gates are shown, with which we can build a digital circuit. To these we normally add the possibility of obtaining a copy of a bit (a wire that splits), an operation we call FANOUT, and the possibility of swapping two bits, called CROSSOVER.

It is possible to show that with this limited set of gates one can compute any function $f : \{0,1\}^n \to \{0,1\}^m$. This is not sufficient to describe a useful computational model. Before seeing which characteristics we require, we define the concept of a circuit family.

A *family of circuits* is a set of circuits $\{C_n\}$ indexed by $n \in \mathbb{N}$. The circuit C_n has n input bits and a finite number of extra work bits and output bits. Given a circuit C_n and a bit string x of maximum length n as input of the circuit, we denote by $C_n(x)$ the output of the circuit.

We say that a family of circuits is *consistent* if all circuits in the family compute the same function $C(\cdot)$. Formally, the definition of consistency is: the family of circuits $\{C_n\}$ is consistent if for $m < n$ and x of length at most m bits, then $C_m(x) = C_n(x)$.

Finally, we define a family of circuits to be *uniform* if there exists an algorithm for a Turing machine that, given n as input, is able to generate a description of C_n. That is, this algorithm must provide the list of gates that constitute C_n and the description of how these gates are connected to each other.

We are finally ready to describe the formal characteristics of the circuit model of computation. In this model we consider functions computable by families of circuits $\{C_n\}$ such that:

- the circuits in the family are *acyclic*: no loops must be present in the circuit, this prevents the occurrence of possible instabilities;
- $\{C_n\}$ is a *consistent* family of circuits: all circuits compute the same function (on inputs of increasing size);
- $\{C_n\}$ is a *uniform* family of circuits: the circuits can be generated from a reasonable algorithm.

It is possible to show that the class of functions computable by a *consistent*, *uniform*, and *acyclic* family of circuits is exactly the same as that computable on a Turing machine.

2.1.2 Computational problems

The theoretical models that we have defined in the previous section (and other equivalent ones that have been proposed in the course of studying computer science) allow us to formally study computational problems. This analysis depends on the answer to three fundamental questions:

WHAT IS A COMPUTATIONAL PROBLEM: by computational problem we mean a specific class of problems called *decision problems*. This choice allows us to isolate a restricted class to simplify the analysis and make progress in the development of the theory. We will see that the results obtained extend well beyond decision problems;

HOW TO DEVELOP AN ALGORITHM TO SOLVE IT: once the problem has been identified, it is necessary to develop an algorithm capable of finding a solution. We are then interested in determining whether there are general algorithms capable of solving large classes of problems. Finally, we would like to find tools to guarantee that an algorithm actually works as expected;

WHAT RESOURCES ARE NECESSARY FOR THE SOLUTION: an algorithm consumes computational resources such as time, space, and energy; we want to know whether the developed algorithm consumes more resources than actually necessary. Moreover, we would like to classify problems according to the resources required to solve them.

Computational complexity

The study of computational complexity aims to quantify the computational resources required to solve a given problem. To study resource consumption independently of the characteristics of the reference architecture[4], *asymptotic notation* is introduced, which captures the essential behavior of a function:

- O ("big O"): used to define the *upper bound* of a function;
- Ω ("big Omega"): used to define the *lower bound* of a function;
- Θ ("big Theta"): is the combination of O and Ω, describing the asymptotic behavior of a function.

Computational complexity considers the spatial and temporal resources required to solve a problem and seeks to demonstrate the existence of a lower bound for these resources regardless of whether an optimal algorithm exists that meets this lower bound. Clearly, the development of algorithms should aim to

4 We mean independence from specific architectural decisions that, for example, allow a processor to save a few clock cycles whenever a certain operation is performed. We will soon see that different computational models can instead have asymptotically different resource consumption.

find an algorithm that does not use more resources than necessary, that is, the theoretical lower bound identified by the study of computational complexity.

In developing the theory of computational complexity, it has been observed that resource consumption depends on the computational model of reference; for example, the three different Turing machines presented in the previous section can solve the same problem with a very different number of operations.

This problem has been clearly addressed by dividing problems into two classes:

- *easy* problems, also called *tractable* or *feasible*, are problems for which there exists an algorithm capable of finding a solution in *polynomial* time with respect to the size of the problem. An algorithm whose resource consumption grows as $O(p(n))$, where n is the size of the problem and $p(\cdot)$ is a polynomial, is called *efficient*;
- *hard*, *intractable*, or *infeasible* problems, if the best algorithm requires exponential resources[5]. In this case we speak of *inefficient* algorithms.

There are several reasons why the choice of dividing problems into two classes of computational complexity (polynomial and exponential) is appropriate. The first is historical: the polynomials describing most proposed algorithms do not have high degree, which means polynomial algorithms almost always outperform exponential algorithms. The second reason is more foundational and is due to the *strong Church–Turing thesis*:

Thesis 2. *Any model of computation can be simulated on a probabilistic Turing machine with at most a polynomial increase in the number of elementary operations required.*

This means that probabilistic Turing machines appear to be the strongest "reasonable" model of computation. Formally, we can state that if, considering a certain computational model (different from the probabilistic Turing machine), it is possible to solve a problem with k elementary operations, then it is possible to solve the same problem on a probabilistic Turing machine with at most $p(k)$ elementary operations (where $p(\cdot)$ is a polynomial).

Reversing this description, the strong Church–Turing thesis states that, given a certain problem to be solved, if there is no efficient algorithm for the probabilistic Turing machine, then an efficient algorithm does not exist for any computational device.

Classes of problems

For now we have divided problems into polynomial and exponential, let us now consider finer divisions of computational problems, taking into account not only the time required but also the space needed to find the solution. Before dividing problems into classes, we formally define the decision problems introduced at the beginning of this section.

A decision problem is a problem whose answer is yes or no. We can define them formally using the terminology of *formal languages*. A language L over the alphabet Γ is a subset of the set Γ^* of all (finite) strings of symbols from Γ.

5 We consider exponential anything that grows faster than a polynomial; $n^{\log n}$, with some abuse of notation, is considered an exponential function.

A string represents the input of our problem and can be written on the tape of a Turing machine. The machine performs a computation that may terminate by writing on the tape a string equivalent to yes or no depending on whether the input string belongs to the language L.

In general, a language L is *decided* by a Turing machine if the machine is able to decide whether an input x on its tape is a member of the language L or not.

P CLASS: We say that a problem can be solved in polynomial time if there exists a Turing machine capable of deciding whether the input x of size n belongs to the language in time $O(n^k)$ for some finite k; we say that this problem belongs to $\mathsf{TIME}(n^k)$. The set of all languages in $\mathsf{TIME}(n^k)$ is denoted by P. P is our first example of a *complexity class*.

To prove that a language L belongs to P, it is sufficient to show an algorithm that decides L in polynomial time. Unfortunately, there exist problems for which such an algorithm has not yet been found. One may then ask whether there exist problems outside P.

Conversely, although there exist non-constructive proofs showing that there must exist problems that require exponential resources, it seems very difficult to prove that a specific problem does not belong to P.

NP CLASS: There exist problems for which a polynomial algorithm has not been found, but if the solution to the problem is somehow provided, it is possible to apply an efficient algorithm to verify such a solution. Problems with this property belong to the class NP. Formally, the set NP contains the languages L such that there exists a Turing machine M such that:

- if $x \in L$ there exists a witness string w such that M halts answering "yes" after a time polynomial in the size of x when the machine starts from the state xbw (b is the blank space);
- if $x \notin L$ for any witness string w, M halts answering "no" after a time polynomial in the size of x when the machine starts from the state xbw

The most famous open problem in computer science is whether or not there are problems in NP which are not in P, often abbreviated as the $\mathsf{P} \neq \mathsf{NP}$ problem.

Within the class of problems NP we find the NP-*complete* problems. A language L is NP-complete if any other language in NP can be *reduced* to L. A language B is said to be reducible to a language A if there exists a Turing machine that, in polynomial time, starting from x produces $R(x)$ and $x \in B$ if and only if $R(x) \in A$. We can view reduction as a translation of one problem into another equivalent one: finding the solution to one means solving the other as well. NP-*complete* problems represent the hardest problems in the class NP, in the sense that finding a lower bound for an NP-*complete* problem means finding a lower bound for all problems in NP.

An NP-*complete* problem is the following: given a circuit with n input bits and one output bit, determine whether there exists an assignment of the input bits such that the output of the circuit is 1. This problem is called the *circuit satisfiability* problem or CSAT. The proof of NP-*completeness* of CSAT is due to the *Cook–Levin theorem*

Theorem 4. *CSAT is NP*-complete.

Once a language L that is NP-*complete* has been found, to prove that another language L' is NP-*complete* it is sufficient to find a reduction from L to L'.

PSPACE CLASS: If we stop considering the time required to solve a problem and instead focus on space consumption, we can define the class of problems PSPACE. This class contains the languages L that can be recognized by a Turing machine using a polynomial number of working bits without time limitations. The classes of problems P and NP are subsets of PSPACE, but there are no proofs that these subsets are strict or not.

A CHAIN OF INCLUSIONS: We can organize the main classes of computational problems in the following chain of inclusions:

$$\mathsf{L} \subseteq \mathsf{P} \subseteq \mathsf{NP} \subseteq \mathsf{PSPACE} \subseteq \mathsf{EXP}. \tag{2.1}$$

Where L are the problems solvable in logarithmic space and EXP the problems solvable in exponential time.

Although most computer scientists believe that each of these inclusions is strict, there are no proofs of this. However, there exist results that allow us to state with certainty that there is not a single class of problems. The *time hierarchy theorem* and the *space hierarchy theorem* ensure respectively that $\mathsf{L} \subset \mathsf{PSPACE}$ and $\mathsf{P} \subset \mathsf{EXP}$. This means that in Equation 2.1 there is at least one strict inclusion, even if we do not know which one.

Approximate solutions

If we need to solve an NP-*complete* problem, today we can rely on many algorithms that perform well on most instances of these difficult problems. If we happen to be particularly unlucky and the instance we are trying to solve does not appear to have favorable properties for our algorithms, we can change approach and search for an *approximate solution*.

There exist NP-*complete* problems for which approximate algorithms are known that guarantee a certain maximum error with respect to the correct solution. There is an entire branch of computer science devoted to approximation algorithms. It is possible to define the class of languages MAXSNP, analogous to the class NP, for which it is possible to efficiently verify an approximate solution to the problem.

BPP CLASS: This is the last class of problems that we introduce considering classical computational models, and it will be useful in the next section where we will introduce quantum models of computation. To define this class of languages we change the computational model; we have already introduced the probabilistic Turing machine and have seen that it is the most powerful classical model of computation.

With this model we can define the class BPP, that is *bounded-error probabilistic time*, which contains the languages L for which there exists a probabilistic Turing machine M such that if $x \in L$ then M accepts x with probability $\geqslant \frac{3}{4}$ and if $x \notin L$ then M rejects x with probability $\geqslant \frac{3}{4}$[6].

6 The value $\frac{3}{4}$ is arbitrary; it is sufficient that the probability be greater than $\frac{1}{2}$.

The *Chernoff bound* implies that with a limited number of repetitions of the algorithm, the probability of success is amplified to the point of being essentially one. For this reason, languages in **BPP**, even more than those in **P**, are considered those that can be solved efficiently on a classical computer.

2.1.3 Quantum computational complexity

Estendiamo ora la classificazione dei problemi computaziuonali considerando un nuovo modello computazionale: i *quantum circuit*. Nella Sezione definiremo con precisione cosa è un *quantum gate* e come si possano assemblare per creare dei quantum circuit. Per il momento ci basta sapere che un quantum gate applica una trasformazione unitaria (introdotta nella Sezione 1.4.2) a uno o più quantum bit (*qubit*).

BPQ Class

Grazie al concetto di quantum circuit possiamo definire la classe di problemi **BQP**: l'iniseme dei problemi decisionali che possono essere risolti with bounded probability of error using a polynomial size quantum circuit. Più formalmente possiamo dire che il linguaggio $L \in$ **BQP** if there is a family of polynomial size quantum circuits which decides the language, accepting strings in the language with probability at least $\frac{3}{4}$, and rejecting strings which aren't in the language with probability at least $\frac{3}{4}$.

In questa definizione possiamo vedere nuovamente il concetto di famiglia di circuiti, questo perhé un quantuym circuit può trovare la soluzione a un problema solo fino a una certa dimensione, se questa aumenta sarà necessario modificare il circuito. Perchè un problema sia in **BQP** richiediamo che:

- la dimensione dei circuiti della famiglia che risolvono il problema cresca al più polinomialmente rispetto alla dimensione della stringa x in input;
- la famiglia di circuti sia *uniforme* ovvero che esista un set di istruzioni chiare, il nostro algoritmo, che generino il quantum circiut.

Relazioni con le classi classiche

One of the most significant results in quantum computational complexity is that **BQP** $\in$ **PSPACE** . It is clear that **BPP** $\in$ **BQP**. Rimane aperta la questione se **BPP** $\neq$ **BQP**, in altre parole non si è ancora trovata una dimostrazione del fatto che un computer quantistico sia più potente di un computer classico.

La disuguaglianza **BPP** $\neq$ **BQP** non ha solo effetti nella comparazione tra modelli di computazione classici e quantistici, ma avrebbe ripercussioni anche sulla gerarchia di inclusioni classiche. Se riscriviamo l'Equazione 2.1 alla luce di **BQP** $\in$ **PSPACE** oteniamo:

$$\mathsf{L} \subseteq \mathsf{P} \subseteq \mathsf{NP} \subseteq \mathsf{BQP} \subseteq \mathsf{PSPACE} \subseteq \mathsf{EXP}. \tag{2.2}$$

Assumendo che **BPP** $\neq$ **BQP** potremmo dedurre che **BPP** $\neq$ **PSACE**.

Esistono forti dubbi che unmdello computazionale quantistico non possa risolvere tutti i problemi in **NP**, in particolare sarebbero esclusi i problemi **NP**-completi. Tuttavia ci sono algoritmi che suggeriscono che un computer quantistico possa risolvere in tempi polinomiali un problema fuori da **P** e da **BPP**.

La classe di probemi che appartengono a **NP** ma non a **NP**-complete sono detti **NPI** (**NP** *intermediate*) . Il problema della fattorizzazioni in numeri primi appartiene a questa classe di problemi ed è risolto dall'algoritmo di Shore in tempi polinomiali. Questo algoritmo sembrerebbe dimostrare la superiorità computazionale del modello quantistico; tuttavia, sfortunatamente, non esiste una dimostrazione di **P** $\neq$ **NPI**. Questa è la classe di problemi su cui i researchers in quantum computation and quantum information puntano per mostrare la superiorità della computazione quantistica over quella classica.

2.2 GATE BASED PARADIGM

Il paradigma di computazione quantum gate based è la diretta implementazione del modello computazionale che abbiamo appena descritto. Questo approccio alla compuazione quantistica si fonda su un grande lavoro teorico, parte del quale abbiamo accennato nella sezione precedente e per questo la maggior parte dei risultati teorici sulla computazione quantistica discendo da questo paradigma.

Come suggerito dal nome questo modello si basa sui quantum gate, i quantum gate manipolano *qubit* applicando loro trasformazioni fisiche. Lo scopo del Capitolo **??** era quello di convincerci che questi gate devono avere determinate caratteristiche per aderire alle legi della fisica.

I quantum gate modificano i *qubit* applicano operatori unitari. Nella Sezione 1.4.2 ci siamo concentrati sul significato fisico di un operatore unitario e abbiamo visto come conoscendo lo stato del sistema dopo l'applicazione dell'operatore potevamo risalire allo stato iniziale.

Ci concentriamo ora sul quali implicazioni abbia un operatore unitario su un modello di computazione. La prima conseguenza, più ovvia è che siamo autorizzati a costruire unicamente gate reversibili, ovvero gate che ci permettano di conoscere lo stato dei *qubit* in ingresso se conosciamo lo stato dei *qubit* in uscita.

2.2.1 Gate reversibili classici

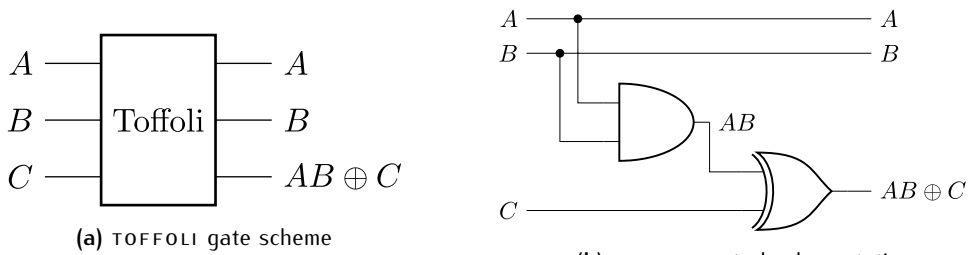
I gate reversibili non esistono solo quando si parla di computer quantisitci. Esistono numerosi esempi di gate classici reversibili e vi sono studi che partono da una computazione classica completamente reversibile per abbattere drasticamente il consumo energetico dei calcolatori. Anche per noi è utile esaminare alcuni gate reversibili classici perche alcune considerazioni che possiamo fare su questi ci torneranno utili nello studio dei quantum gate.

Il più semplice gate reversibile è la porta NOT (Figura 2.1a). Un'altro importante gate reversibile classico è il TOFFOLI gate mostrato in Figura 2.2[7]. Se osserviamo la tabella di verià del TOFFOLI gate (Figura 2.3) possiamo notare come gli input siano univocamente identificati dagli output (definizione di gate reversibile). Il TOFFOLI gate ha tre bit in ingresso, la tabella di verità ha dunque $2^3 = 8$ righe, siccome gli output devono essere delle chi-

A	B	C	A	B	$AB \oplus C$
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	1
1	1	1	1	1	0

Figure 2.3: TOFFOLI gate true table

7 La Figura 2.2b mostra l'implementazione del TOFFOLI gate usando i gate mostrati in Figura 2.1.



(a) TOFFOLI gate scheme

(b) TOFFOLI gate Implementation

Figure 2.2: Circuit sampling

avi univoche non ci sono ripetizioni negli output. Questo significa che anche negli output ci sono tutte le combinazioni possibili di tre bit. Possiamo allora dedurre che il TOFFOLI gate agisce *permutando* le righe in ingresso della tabella di verità. Questo è vero in generale:

Theorem 5. *La colonna di output della tabella di verità di un gate reversibile classico è una permutazione della colonna degli input.*

Questo concetto di permutazuione dell'input tornerrà anche quando parleremo di gate quantistici.

2.2.2 Quantum gate

Nel Capitolo 2.5 abbiamo visto che lo spin è un esempio di proprietà binaria e avevamo classificato come *qubit* tutte quelle proprietà fisiche che misurate restituiscono solo due valori. Avevamo misurato lo spin rispetto agli assi coordinati e avevamo individuado diversi stati di spin (*up* and *down*, *left* and *right*, *in* and *out*).

Questi stati cambiano leggermente notazione quando passiamo dallo studio della fisica alla computer science. Prima di descrivere i quantum gate introduciamo questa notazione alternativa per gli stati di un *qubit* e mostriamo una rappresentazione grafica che ci tornerà utile per capire l'effetto dei quantum gate.

qubits

Nella computer science i gli stati *up* and *down* (Equazioni 2.5 e 2.5) rappresentano rispettivamente un *qubit* spento $|0\rangle$ e acceso $|1\rangle$. Denotiamo con $|+\rangle$ e $|-\rangle$ i *qubits* negli stati *right* and *left* (Equazioni 2.5 e 2.5); infine gli stati *in* and *out* (Equazioni 2.5 e 2.5) sono rappresentati da $|i\rangle$ e $|-i\rangle$. Possiamo riscrivere le equazioni citate alla luce della nuova notazione:

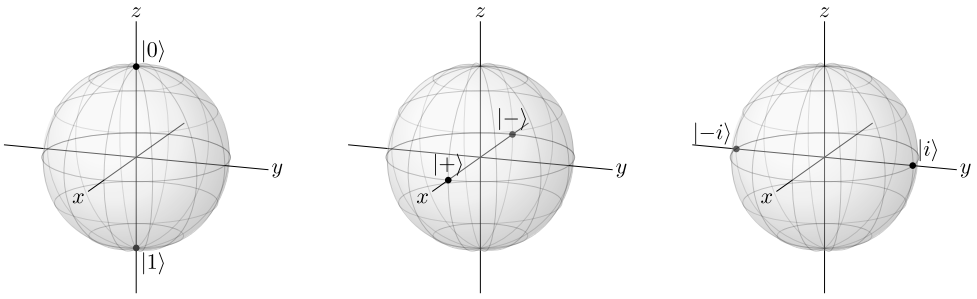
$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \qquad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \tag{2.3}$$

and:

$$|i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i\,|1\rangle) \qquad |-i\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i\,|1\rangle) \tag{2.4}$$

Per rappresentare i *qubits* in un modo grafico che rispetti le proprietà matematiche delgi stati quantistici possiamo ricorrere a una sfera detta *sfera di Bloch*.

La sfera ha raggio 1 e ogni punto sulla sfera rappresenta un *qubit*[8]. La Figura 2.5 mostra, sulla sfera di Bloch, i *qubits* descritti sopra.



(a) States $|0\rangle$ and $|1\rangle$ **(b)** States $|+\rangle$ and $|-\rangle$ **(c)** States $|i\rangle$ and $|-i\rangle$

Figure 2.4: Bloch Sphere **[bibid]**

quantum gate

Ricordando i risultati raggiunti nel Capitolo 2.5 e considerando anche quanto abbiamo detto riguardo i gate cclassici reversibili nella Sezione 2.5 possiamo fare le seguenti affermazioni riguardo i quantum gate:

- i quantum gate sono mappe lineari che mantengono la probabilità totale pari a 1[9];
- possiamo rapptresentare un qwuantum gate come una matrice che mantiene la probabilità totale pari a 1;
- i quantum gate sono matrici unitarie e le matrici unitarie sono quantum gate;
- un quantum gate $\mathbf{U}$ è reversibile e $\mathbf{U}^\dagger$ è il gate inverso;
- i gate reversibili classici sono quantum gate validi[10].

Ora che abbiamo descritto le principali proprietà dei quantum gate possiamo esaminare l'effetto che questi gate hanno sui *qubits*, cominciamo la nostra analisi dai gate che operano su un singolo *qubit*.

SINGLE QUBIT QUANTUM GATE: I quantum gate che operano su un singolo *qubit* possono essere visualizzati come delle *rotazioni* sulla sfera di bloch. Qualsiasi rotazione può essere percorsa sia in avanti che indietro, questo rende le rotazioni reversibili e di conseguenza dei quantum gate validi. Vediamo ora alcune rotazioni che definiscono gate particolarmente interessanti.

Rotazioni di 180° rispetto agli assi definiscono i *gate di Paoli*:

- Paoli gate X: rotazione rispetto all'asse x. Questo gate è la versione quantistica del gate classico NOT:

$$X|0\rangle = |1\rangle \qquad X|1\rangle = |0\rangle\,;$$

8 A meno di una fase globale, che però è ininfluente.

9 Ricordiamo che la probabilità totale dello stato $|s\rangle = c_1|0\rangle + c_2|1\rangle$ formato da un *qubit* è $\alpha^2 + \beta^2$

10 In paricolare è possibile mostrare che un gate reversibile classico permuta le ampiezze di probabilità, quindi mantiene la probabilità totale pari a 1

- Paoli gate Y: rotazione rispetto all'asse y. Questo gate non ha un corrispettivo classico e, applicato ai *qubits* $|0\rangle$ e $|1\rangle$ li modifica come segue:

$$Y|0\rangle = i|1\rangle \qquad Y|1\rangle = -i|0\rangle\,;$$

- Paoli gate Z: rotazione rispetto all'asse z. Anche questo è un gate puramente quantistico, che agisce nel seguente modo:

$$Z|0\rangle = |0\rangle \qquad Z|1\rangle = -|1\rangle\,;$$

- Paoli gate I: rotazione nulla. Questo gate non modifica lo stato dei *qubits*:

$$I|0\rangle = |0\rangle \qquad I|1\rangle = |1\rangle\,.$$

È possibile definire questi gate anche come matrici, la definizione è uguale a quella degli operatori che rappresentavano le ossevabili σ_x, σ_y e σ_z (rispettivamente equazioni **??**, **??** e **??**) più la matrice identità per il gate I. Se infatti osserviamo qle matrici che descrivono le osservabili notiamo che oltre ad essere operatori Hermitiani (che descrivono osservabili) sono anche Unitari, quindi dei gate quantisitici validi.

Possiamo poi considerare rotazioni inferiori ai 180°, definiamo allora il gate S o *phase gate* come rotazione di 90° rispetto all'asse z e dimezzando ancora l'angolo di rotazione otteniamo il gate T (rotazione di 45° rispetto a z). L'azione di T su $|0\rangle$ e $|1\rangle$ è:

$$T|0\rangle = |0\rangle \qquad T|1\rangle = e^{i\frac{\pi}{4}}|1\rangle\,.$$

L'ultimo gate che agisce su un singolo *qubit* che consideriamo è la *porta di Hadamard* (H). Questo gate può essere interpretato come una rotazione di 180° rispetto all'asse $x+z$. H permette di creare sovrapposizione di stati[11], in particloare:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle \qquad H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle$$

La Figura 2.5 mostra i gate descritti come rotazioni sulla sfera di Bloch.

MULTIPLE QUBITS QUANTUM GATE: Quando descriviamo stati formati da più *qubits* dobbiamo descriverli come *prodotto tensoriale* $\otimes$. Ad esempio uno stato in cui due *qubits* siano entrambi nello stato $|0\rangle$ si indica:

$$|0\rangle \otimes |0\rangle \text{ abbreviato in } |0\rangle|0\rangle \text{ abbreviato in } |00\rangle$$

Con due *qubits* la base z è $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$. Un generico stato $|s\rangle$ sarà la sovrapposizione degli stati della base

$$|s\rangle = c_1|00\rangle + c_2|01\rangle + c_3|10\rangle + c_4|11\rangle\,,$$

e la probabilità totale sarà $|c_1|^2 + |c_2|^2 + |c_3|^2 + |c_4|^2$ che, se lo stato è normalizzato, sarà uguale a 1.

Alcuni stati composti possono essere scritti come prodotto tensoriale di stati di *qubits* singoli, ad esempio:

$$\frac{1}{2}\cdots.$$

11 Torneremo su questo nella Sezione 2.5

Questi stati sono detti *product states* o *simply separable states*. Esistono stati che non possono essere scomposti in stati semplici. Si parla allora di *entangled states*, degli stati in cui la completa conoscenza del vettore di stato del sistema completo non ci da nessuna informazione sugli stati degli elementi che compongono il sistema[12].

Studiamo i multiple qubits quantum gate proprio per la loro capacità di creare stati entangled. Il primo gate che permette l'entanglement tra *qubits* è il CNOT o cX.

Il gate CNOT (*controlled* NOT) è un gate a due *qubit*, il primo è detto di *controllo*, il secondo è il *qubit target*. Se il *qubit* di controllo è settato il *qubit* target è invertito (da questo deriva il nome controlled NOT). Il comportamento del gate CNOT può essere descritto come:

$$\mathrm{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Figure 2.5: CNOT gate

$$\mathrm{CNOT}\,|a\rangle\,|b\rangle = |a\rangle\,|a \oplus b\rangle\,.$$

In Figura 2.5 è riportata la definizione matriciale del CNOT. Questo gate è in grado di creare stati entangled. In particolare:

content...

Gli stati $|\Psi\rangle^+$, $|\Psi\rangle^-$, $|\Phi\rangle^+$ e $|\Phi\rangle^-$ non possono essere ottenuti come prodotto tensoriale di stati di *qubits* singoli e sono quindi stati entagled. Questi quattro stati sono detti *Bell states*, sono ortogonali e formano una base detta *Bell basis*.

Altri quantum gate importanti sono quelli che abbiamo già incontrato palrlando di gate classici, abbiamo già visto che un gate classico reversibile è un quantum gate valido. I gate che ci interessano di più sono il TOFFOLI gate e il CROSSOVER gate.

Non è invece possibile costruire un alternativa quantistica del FANOUT gate, quindi non possiamo ottenere una copia di un *qubit* senza sapere il suo stato. Misurare lo stato, però, vorrebbe dire inevitabilmente modificarlo, quindi in generale non possiamo duplicate un *qubit*. Questo è un risultato teorico importante e prende il nome di *non-cloning theorem*:

Theorem 6. *It is impossible to make a copy of an unknown quantum state. Qubits cannot be copied.*

Universal quantum gates

Come esistono dei set di gate classici che permettono di costruire quealsiasi altro gate[13] possiamo individuare dei set di quantum gate universali. Un set di quantum gate universali deve essere in grado di:

- creare sovrapposizione di stati come la porta di Hadamerd H;
- creare stati entangled come il gate CNOT;

12 L'entanglement è uno degli aspetti più interessanti della meccanica quantistica. Discuterne approfonditamente le implicazioni fisiche è furi dallo scopo di questo lavoro. Si raccomanda la lettura di [**bibid**].

13 Esempi di questi set sono: { AND, NOT }, { OR, NOT }, { NAND } e { TOFFOLI }.

- creare ampiezze di probabilità complesse, H e CNOT contengono solo valori reali, il phase gate S invece consente di ottenere ampiezze di probabilità complesse.

Nonostante il set di gate che abbiamo implicitamente definito $\{$ CNOT, H, S $\}$ abbia tutte le proprietà richieste è possibile mostrare che non è un set universale. In particolare il *Gottesman-Knill theorem* avverma che:

Theorem 7. *A quantum circuit containing only* CNOT, H *and* S *gates is efficiently simulated by a classical computer.*

Chiamiamo i circuiti generati da questo set di gates *Clifford group*, e il set $\{$ CNOT, H, S $\}$ prende, di conseguenza, il nome di *Clifford group generator*. Un set di gates universali deve quindi generare più del *Clifford group*, anche se non è stato dimostrato che sia condizione sufficiente a essere un set universale.

Esempi di set di gates universali sono:

- $\{$ CNOT, H, T $\}$: rimpiazzare il phase gate con il gate T nel *Clifford group generator* ci permette di ottenere un set di porte universali;
- $\{$ TOFFOLI, H, S $\}$: partiamo nuovamente da *Clifford group generator* e otteniamo un set universale sostituendo al CNOT il TOFFOLI gate;

Il concetto di set di gates universali quando si parla di quantum computing è differente da quello a cui siamo abituati con le porte classiche. Nel caso classico possiamo costruire una qualsiasi tabilla di verità partendo da un set di gate classici universali. Il significato di set universale nella computazione quantistica invece è descritto dal *Solovay-Kitaev theorem*:

Theorem 8. *Any universal gate set can approximate a quantum gate on* n *qubits to precision* ϵ *using* $\Theta(2^n \log^c(\frac{1}{\epsilon}))$ *gates for some constant* c

Non ci deve stupire il termine 2^n perchè un operatore unitario che agisce su n *qubits* (cioè il gate che stiamo cercando di approssimare) è una matrice di dimensione $2^n \times 2^n$. Ciò che ci intgeressa davvero è il termine $\log^c(\frac{1}{\epsilon})$ che ci dice che possiamo ridurre l'errore di approssimazione ad un costo che crese come il polinomio di un logaritmo —questa funzione è spesso chiamata *polylog*— in modo, cioè, efficiente.

2.2.3 Algoritmi

Abbiamo discusso cosa siano i quantum gate e come essere assemblati per costruire un quantum circuit. Vediamo ora come questo circuito deve essere fatto per poter essere utile alla risoluzione di un problema computazionale. In questa sezione introduciamo i circuiti basati su oracolo quantististico e presentiamo il più semplice algoritmo basato su oracolo.

Quantum Oracles

un oracolo è una funzione booleana, ovvero una funzione che abisce su dei *qubits* e che si può comporre con dei gate. Per poter essere tradotta in quantum gate dobbiamo fare in modo che il gate ottenuto sia reversibile. È possibile, in generale, rendere reversibile un circuito usando un extra *quibit* e applicando il gate XOR tra l'output del circuito e il *qubit* extra.

Se a esempio se la funzione $f(x)$ agisce su un solo *qubit* possiamo costruire un gate reversibile $\mathbf{U}_f$ come quello mostrato in Figura 2.6. È facile vedere che a prescindere dll'azione di $f(x)$, che può essere reversibile o meno, conoscendo gli output del gate è possibile risalire agli input. Possiamo riscrivere l'applizione di $\mathbf{U}_f$ ai *qubits* come:

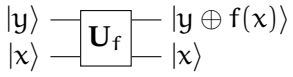


Figure 2.6: Quantum oracle gate

$$|x\rangle\,|y\rangle \xrightarrow{\mathbf{U}_f} |x\rangle\,|y \oplus f(x)\rangle$$

L'extra *qubit* è detto *answer qubit* o *target qubit*, $|x\rangle$ è detto *input qubit*. Quando settiamo $|y\rangle = |0\rangle$ possiamo leggere nel target *qubit* il risutato di $f(x)$ applicata a $|x\rangle$.

In realtà questo non è il modo standard di usare un quantum oracle. Ciò che si fa è preparare il target *qubit* nello stato $|-\rangle$ poi applocare il gate $\mathbf{U}_f$. I quantum gate possono ovviamente essere applicati anche a stati sovrapposti, e in questo caso il risultato è un cambiamento della fase del *qubit* $|x\rangle$ mentre il *qubit* target rimane invariato. Possiamo scrivere l'applicazione di questo gate come:

$$|x\rangle\,|-\rangle \xrightarrow{\mathbf{U}_f} \begin{cases} |x\rangle\,|-\rangle & \text{if } f(x) = 0 \\ -\,|x\rangle\,|-\rangle & \text{if } f(x) = 0 \end{cases} = (-1)^{f(x)}\,|x\rangle\,|-\rangle = (-1)^{f(x)}\,|x\rangle$$

Dove, nell'ultimo passaggio, tralasiamo il valore del *qubit* terget perhè non viene modificato dal gate.

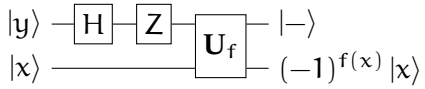


Figure 2.7: Phase oracle

Siccome usato in questo modo l'oracolo applica un cambiamento di fase a $|x\rangle$ viene detto *phase oracle*. La Figura 2.7 mostra l'applicazione del phase oracle, in particolare si nota come sia possibile preparare $|y\rangle$ nello stato $|-\rangle$ per poter poi usare $\mathbf{U}_f$ come phase oracle[14].

Deutsch' algorithm

Questo algoritmo mostra come usare il phase oracle per risolvere un problema computazionale. Vedremo che già con questo primo algoritmo otteniamo performance migliori rispetto a quello che potremmo fare con un architettura classica. In questo caso però l'incremento prestazionale è polinomiale, questo quindi non è uno di quegli algoritmi che suggeriscono che i computer quantistici possano risolvere in modo efficiente una classe di problemi più ampia dei calcolatori classici[15].

Il problema computazionale che vogliamo risolvere è quello di stabilire la *parità* di due bit b_1 e b_2 ignoti. Stabilire la parità significa calcolare $b_1 \oplus b_2$. Per ottenere il risultato abbiamo a disposizione un oracolo $f(x)$ con $x \in \{0, 1\}$ che, interrogato con l'indice del bit ignoto, ci fornisce il valore del bit, cioè:

$$f(0) = b_0 \qquad f(1) = b_1$$

14 È possibile ottenere lo stesso risultato applicando prima il Paoli gate X poi la porta di Hadamard H

15 Mostrare questi algoritmi, tra cui l'algoritmo di Shore è fuori dallo scopo di questo lavoro.

In un modello di computazione classico dobbiamo interrogare l'oracolo due volte, ottenere il valore dei bit e calcolare il risultato dello XOR tra i due valori.

L'algoritmo di Deutsch ci permette di ottenere il valore di parità interrogano l'oracolo una volta sola. La Figura 2.8 mostra come è implementato il quantum circuit.

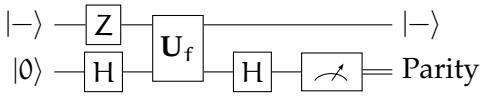


Figure 2.8: Deutsch' algorithm

Senza sviluppare tutti i passaggi algebrici vediamo cosa succede all'input *qubit* $|0\rangle$ man mano che attraversa i quantum gate. Il primo quantum gate è la porta H, l'applicazione di questa modifica $|0\rangle$ in

$$|0\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}|0\rangle + |1\rangle = |+\rangle\,.$$

Ora interroghiamo l'oracolo con un input in sovrapposizione di stati. Il risultato dell'interrogazione è

$$\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \xrightarrow{U_f} \frac{1}{\sqrt{2}}\Big[(-1)^{f_0}|0\rangle + (-1)^{f_1}|1\rangle\Big].$$

Ignorando alcuni passaggi possiamo semplificare questa espressione in:

$$\begin{cases} (-1)^{b_0}|+\rangle & \text{if } b_0 = b_1 \\ (-1)^{b_0}|-\rangle & \text{if } b_0 \neq b_1 \end{cases}.$$

Dopo l'oracolo, se $b_0 = b_1$ otteniamo lo stato $|+\rangle$ più una fase globale, se $b_0 \neq b_1$ otteniamo lo stato $|-\rangle$ più una fase globale. Questa fase non modifica la posizione del *qubit* sulla sfera di Bloch, possiamo già distinguere i due stati con una misura parallela all'asse x. Dato che per convenzione le misure vengono fatte rispetto all'asse z applichiamo ancora una volta la porta H ottenendo

$$\begin{cases} (-1)^{b_0}|0\rangle & \text{if } b_0 = b_1 \\ (-1)^{b_0}|1\rangle & \text{if } b_0 \neq b_1 \end{cases}.$$

L'atto di misura scarta la fase globale e ci permette di distinguere con certezza i due casi: otterremo $|0\rangle$ se i bit incogniti erano uguali (parità pari), $|1\rangle$ se i bit incogniti erano diversi (parità dispari). È interessante notare che durante tutto l'algoritmo non siamo mai venuti a conoscenza del valore dei singoli bit, ma abbiamo ricavato solamente se erano uguali o meno.

L'algoritmo di Deutsch permette di dimezzare il nuoreo di interrogaziooni all'oracolo, e può essere esteso a un numero arbitrario di bit di cui si vuole conoscere la parità. Questo esempio conclude la nostra discussiaone sul paradigma quantum gate based. Troveremo nuovamente questo paradigma nel Capitolo **??** dove lo mischieremo ai concetti di quantum annealing che illustriamo adesso.